Read from a text file

Motivation of reading from a file: initialize variables

- Assign a value directly
  - int size = 3;

    string greeting = "Hello"; //same as string greeting("Hello");
  - Simple. However, to change its value, need to modify the source code.
- Enter a value interactively from a console
  - cout << "Enter your name: ";

    string name;

    cin >> name;

    //Can I enter a full name separated by a space to name?
  - Pro: can enter from console, but need to input each time
- Read a file: Pro: can read a batch of data

Text file

- Only work on plain text file ended by txt, opened in plain text editor like VS code (windows / Mac), notepad++ (windows) , TextEdit (Mac) and vi.
- Typical data file TSV and CSV (not CVS) are opened in Excel.
  - concentrate on TSV in today's class.
- TSV stands for Tab Separated Value, using the Tab character to delimit data.

side note: class = data + operations on data

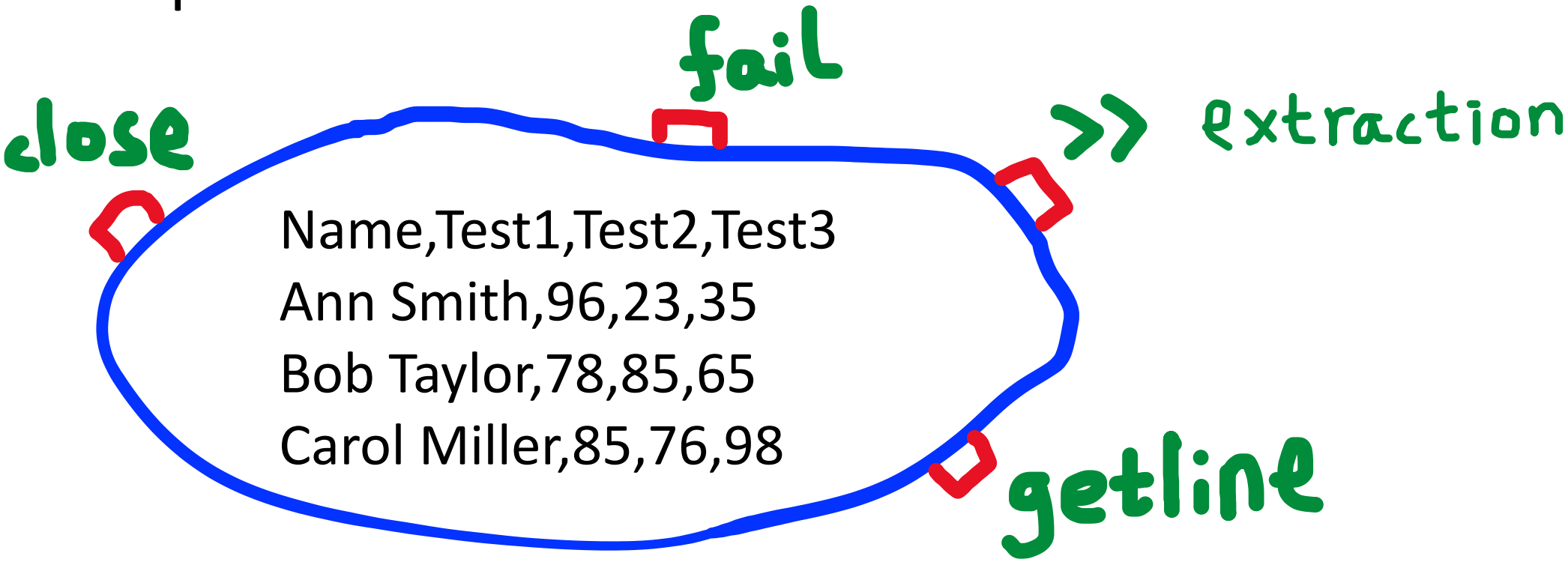
- A hamburger is data (meat-, veg-, bread- layer) and operations on those data.

Person class

- objects of class: any person, for example, you, me
  - Person ann(“Ann Smith”, 18); //ann is an object of Person class, representing a person named ”Ann Smith” whose age is 18.
  - int i; // i is a variable of int type.
- Data members: things that every object of that class know.
  - Data members of Person class are name and age.
- Operations: methods on the data members
  - getName: get the name of current object, ann.getName() returns ”Ann Smith”
  - getAge: get the age of the current object, ann.getAge() returns 18
  - changeName: change the name of the current object, ann.changeName(”Ann Smith Wood ”); change the name of ann of “Ann Smith Wood”.
  - changeAge: change the age of the current object, ann.changeAge(26); changes the age of ann to be 26

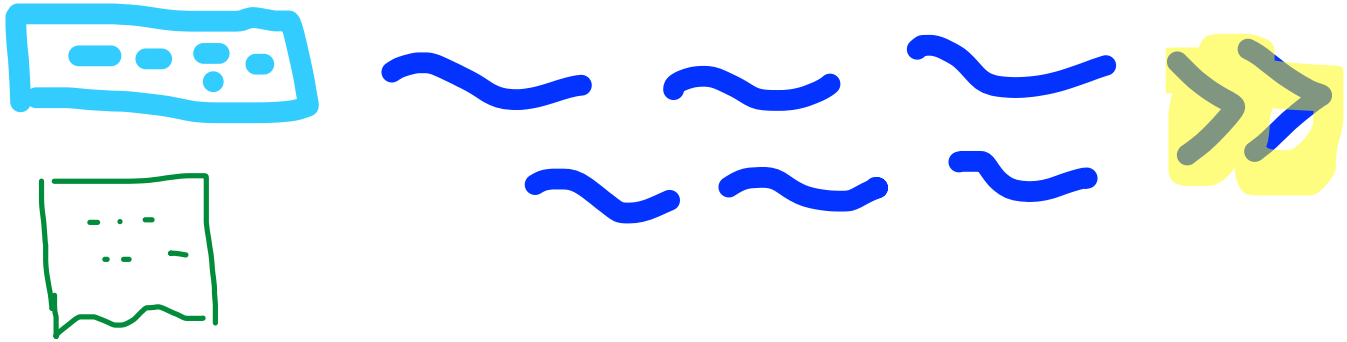
ifstream is a class

- data and operations on those data



read from a file

- Use ifstream class, need to include <fstream>, where f means file.
- Think fstream as data (string, int, floating-point number) streaming from a file.
- Related: cin is standard input stream, data streaming from standard input device, ie, keyboard buffer.



use ifstream

- Create an ifstream object, use a valid variable name. For example, fin.
- **ifstream** fin("data.txt");
- Warning: cannot write as ~~**ifstream** fin = "data.txt";~~
- ifstream is not a primitive data type.
  - example of a primitive data type is **int**

  int count = 3;
- ifstream is a class.
  - example of a class is string.

  string greeting = "Hello"; is same as

  string greeting("Hello");

string is an exception that its objects can be

- initialized as if it were a primitive type or
- instantiated as an object

Steps to read a text file

- Create an ifstream object
  - **ifstream** *objName*(*fileName*);
  - You may name *objName* as fin, or in_file, or whatever valid variable names.
- Check whether the file can be opened or not. If not, report error and exit.
- Now if the file does not fail, the file associated by *objName* can be opened.

Steps to read a text file: II

- Check the structure of the file, you use *objName* as if it were cin.
  - cin is created and associated with standard input stream, no setting is needed.
  - Use getline(*objName*, stringName); to read a line to stringName
  - Use *objName* >> variableName; to get the data to a variable (can be int, double, string, etc) to the corresponding variableName.
- Do not forget to close the file by *objName*.close();

How to read a file: Check contents of the file

Contents of scores.tsv (tab separated values)

```
Name	Test1	Test2	Test3
Ann	96	23	35
Bob		78	85	65
Carol	85	76	98
```

Read data: getline vs cin

- getline(streamObject, stringVariable) reads a line from streamObject, put the result in a string variable
  - The first parameter of getline is an input stream object, it can be standard input object cin or an instantiated ifstream object.
  - The second parameter of getline must be a string variable.

getline vs >> : II

- getline(streamObject, stringVariable, delimiter) reads an item before the delimiter from input, put the result in a string variable
- streamObject >> variable; read a word from the streamObject and try to convert the input to the type of the variable following >>.
  - >> stops at the first white space, that is, space, tab, or newline character.

Read from keyboard vs read from a text file

- To read from the keyboard, use iostream library.
- cin is the standard input device, say, keyboard. Since the keyboard is unique, there is no need to instantiate it.
  - Similarly, there is only one earth; no need to create it.

- To read from a file, use fstream library.
- Text file has only data, no format.
  - Some formats are txt, csv (comma-separated values), tsv (tab-separated values).
- To create a file object that reads a text file called data.txt, use

```
ifstream fin("data.txt");
```

Read from keyboard vs read from a text file: II

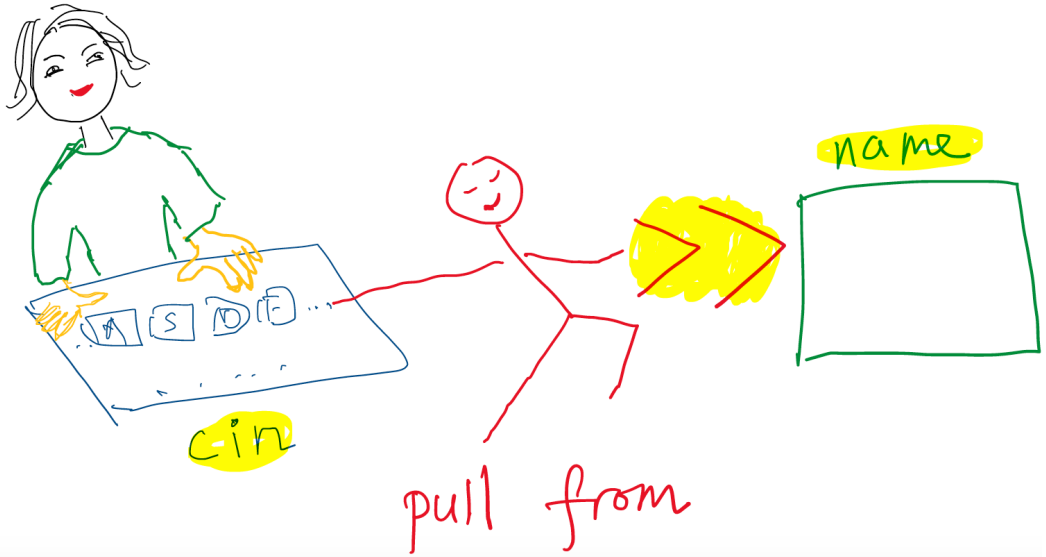
- The keyboard can always be used to input. NO NEED to test it can be opened or not.

- Test whether the file associated with data.txt can be opened or not

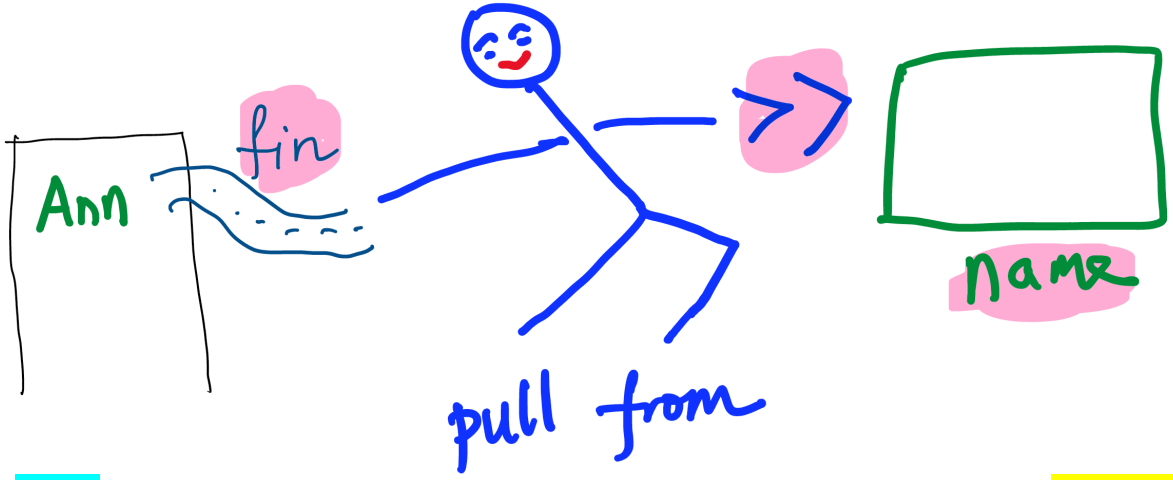
```
if ( fin.fail() ) {
    cerr << "Cannot open file"
            << endl;
    exit(1);
}
```

Read from keyboard vs read from a text file: III

- Depending on the request of interactive input, we may enter a word from keyboard.



- Depending on the structure of the file, we may read a word from keyboard.



fin: name of an ifstream object, may have a different name

Read from keyboard vs read from a text file: IV

```
int num;
cin >> num;
string word;
cin >> word;
string sentence;
getline(cin, sentence);
```

extract an integer from keyboard and put it in num.

```
int num;
fin >> num;
string word;
fin >> word;
string sentence;
getline(fin, sentence);
```

extract an integer from fin and put it in num.

Read from keyboard vs read from a text file: V

No need to close cin object.

When fin is associated with a file, need to close fin by calling

```
fin.close();
```

to close the file.

Example to read from a file

- Given a file called name_age.txt with the following contents

```
name age
Ann 18
Bob 20
Charles 17
```

Key step 1: import necessary library link

```
#include <iostream>
#include <fstream> //ifstream, ofstream
#include <string>

int main() {
    std::cout << "Enter a file name: ";
    std::string fileName;
    std::cin >> filename;
```

Key step 2: instantiate an ifstream object link

```
std::ifstream fin(fileName);
//instantiate an ifstream object fin to read fileName
//The above statement is similar to
//string greeting("Hello);
//instantiate a string object called greeting
//with a sequence of characters "Hello"
```

Key step 3: handle when file cannot be opened link

```
if (fin.fail()) {
    //a file may not be able to be opened
    //for example, unauthorized access, or corrupted file
    std::cerr << fileName << " cannot be opened" <<
std::endl;
    exit(1);
}
```

Key Step 4: use file object as if it were cin link

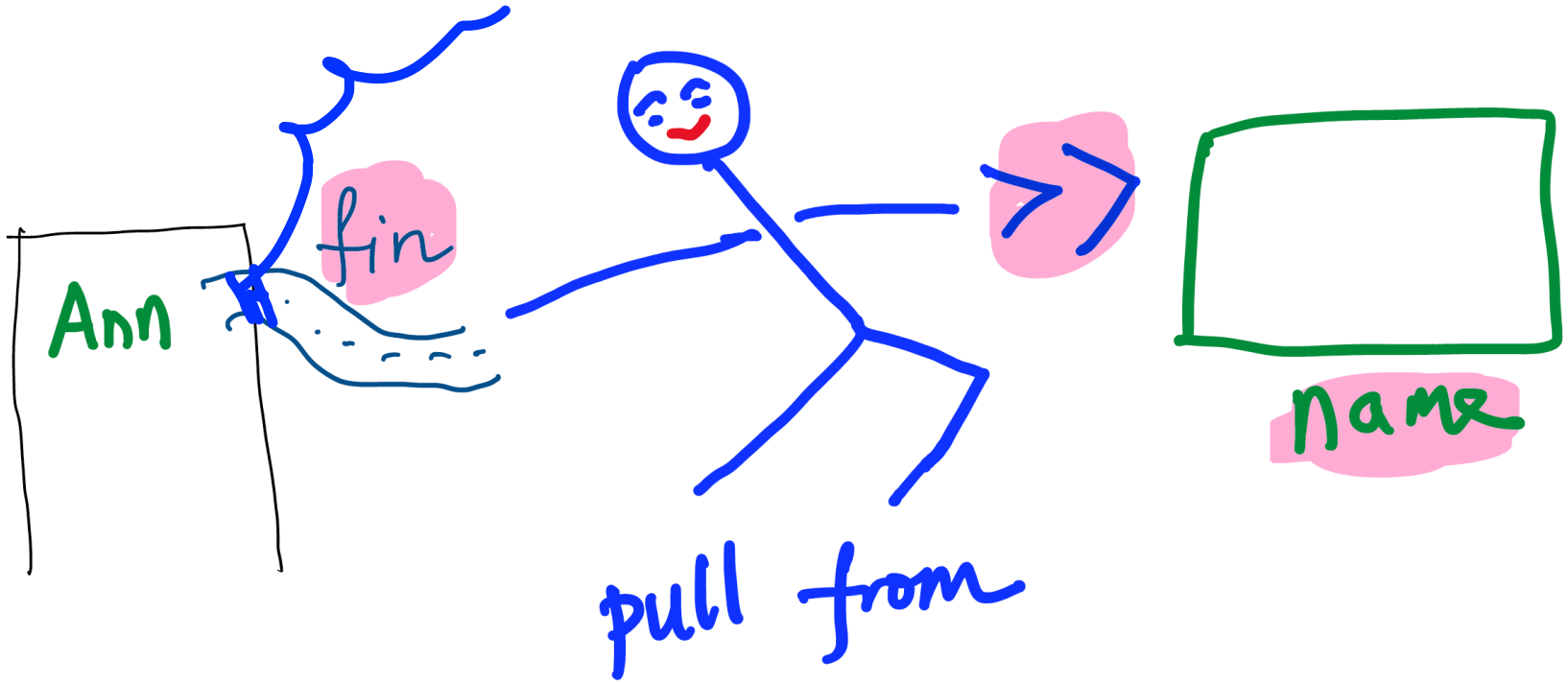
```
//suppose the content of the file is
//name age
//Ann 18
//Bob 20
//Charles 17
std::string caption;
//The only difference is to read from a file vs. read from keyboard
getline(fin, caption); //skip the line with caption
```

Key Step 5: read the data link

```
//read the data, the first column is name,
//the second column is age
std::string name;
int age;
while (fin >> name >> age) {
    //there is only one statement in loop body,
    //it is optional but encouraged to add { and } around the
loop statement
    std::cout << name << "'s age is " << age << std::endl;
}
```

Key Step 6: Do not forget to close the file object link

```
  fin.close();
  //WARNING: do not forget to close a file
  //when it is no longer needed
  return 0;
}
```



Template to read from a text file

```
#include <iostream>
#include <fstream>
#include <climits>
#include <string>

int main() {
    ifstream fin("data.txt");
```

Template to read a text file: II

```
    if (fin.fail()) {
        std::cerr << "file cannot be opened" << std::endl;
        exit(1);
    }

    //TODO: read file ...

    fin.close();
    return 0;
}
```

Summary: use ifstream to read from a file

- Instantiate an ifstream object. It is like to build a pipe to extract the data stored in filename, then data from filename stream out.

```
ifstream fin(filename);
```

- Test whether fin fails or not, if fails, print error message and exit the function.
- Use fin as if it were cin, whatever cin can do, fin can also do. The only difference is that cin reads from the keyword buffer while fin reads from a file.
- Pay attention to the file structure when reading its contents.
- Do not forget to close fin if this file is no longer needed.